

UNIVERSIDAD DE CHILE
FACULTAD DE CIENCIAS FÍSICAS Y MATEMÁTICAS
DEPARTAMENTO DE CIENCIAS DE LA COMPUTACIÓN

GENERACIÓN DE MALLAS DE POLIEDROS MEDIANTE CAVIDADES

TESIS PARA OPTAR AL GRADO DE
MAGÍSTER EN CIENCIAS, MENCIÓN COMPUTACIÓN

MEMORIA PARA OPTAR AL TÍTULO DE
INGENIERO CIVIL EN COMPUTACIÓN

VICENTE IGNACIO GONZÁLEZ BECERRA

PROFESORA GUÍA:
NANCY HITSCHFELD KAHLER

PROFESOR CO-GUÍA:
SERGIO SALINAS

MIEMBROS DE LA COMISIÓN:
PENDIENTE

SANTIAGO DE CHILE
2026

RESUMEN DE LA TESIS PARA OPTAR AL TÍTULO DE INGENIERO CIVIL EN COMPUTACIÓN Y MAGÍSTER EN CIENCIAS, MENCIÓN COMPUTACIÓN
POR: VICENTE IGNACIO GONZÁLEZ BECERRA
FECHA: 2026
PROF. GUIA: NANCY HITSCHFELD KAHLER

GENERACIÓN DE MALLAS DE POLIEDROS MEDIANTE CAVIDADES

Tabla de Contenido

1. Introducción **1**

1.1. Problema 1
1.2. Hipótesis 2
1.3. Preguntas de investigación 2
1.4. Objetivos 2
1.5. Estructura de la Tesis 3

2. Estado del arte **4**

2.1. Generación de mallas de tetraedros 4
2.2. Generación de mallas de poliedros 8
2.3. Estructura de datos 9
2.4. Cavidades 9

3. Trabajo anterior **10**

3.1. POLYLLA 2D 10
3.2. POLYLLA 3D 10
3.3. Criterios de unión 10

4. Algoritmo de cavidades **11**

4.1. Arquitectura 11
4.2. Visualización 11

5. Experimentos y resultados **12**

6. Conclusión **13**

Bibliografía **14**

Índice de Tablas

Índice de Ilustraciones

Figura 1 Un PLC es el conjunto de vertices, segmentos y caras que definen la geometría del dominio, extraído de [1] (Figura 1a) 5
Figura 2 Comparación entre las mallas generados por CGAL con la de TetWild (ours), extraído de [2] (Figura 16) 6
Figura 3 Creación de una *Terminal-edge region* 9

Capítulo 1

Introducción

La generación de mallas es un área que ha estado en el foco de atención constantemente desde el asentamiento de la computación como disciplina. Tanto a nivel académico como industrial, el discretizar un espacio, 2D o 3D, en primitivas de manera automática es clave para poder efectuar simulaciones avanzadas. Bajo el contexto de poliedros, comúnmente se reconocen 2 formas de generar estas mallas: de manera directa, la cual toma información espacial directa, como una nube de puntos o un dominio geométrico, y genera la estructura datos; y la manera indirecta, que toma una malla de tetraedros previamente construida y le aplica una serie de transformaciones para generar una malla de poliedros manteniendo su características en rasgos generales. El presente trabajo se centrará en esta última forma.

Actualmente, existe un algoritmo en dos dimensiones capaz de generar una malla de polígonos a partir de una malla de triángulos conocido como POLYLLA [3]. Tomando este como inspiración se desarrollo una versión tridimensional, donde posteriormente se realizó un estudio en profundidad de la viabilidad y calidad de dichas mallas.

1.1. Problema

Debido al creciente interés y consolidación de nuevos métodos numéricos, como el método de elementos virtuales [4] (*VEM* por sus siglas en inglés), se necesita disponer un algoritmo que sea escalable con respecto a la cantidad de células (tetraedros) mientras contenga propiedades deseables para que dicho método funcione correctamente. No obstante, el estudio realizado sobre POLYLLA 3D dio resultados poco satisfactorios en este ámbito, donde se concluyó que uno de los algoritmos, POLYLLA 3D Face, era capaz de obtener buenos resultados, pero que la ausencia de *kernel* en ciertos poliedros podría afectar su usabilidad para VEM [5]. Con esto en mente se propone una nueva forma de crear poliedros, alejada de la forma tradicional de unir triángulos por sus aristas en POLYLLA 2D: usar cavidades. En este trabajo, las regiones candidatas se definen a partir de la circunsfera asociada a cada tetraedro semilla, es decir, la esfera que pasa por sus cuatro vértices. Esta

debe distinguirse tanto de la esfera inscrita, tangente a las cuatro caras, como de la cavidad de Bowyer–Watson, que es un conjunto de tetraedros en conflicto respecto de un punto de inserción [1].

1.2. Hipótesis

Es posible diseñar e implementar un algoritmo que genere mallas de poliedros a partir de una malla de tetraedros utilizando las cavidades de dichos tetraedros, seleccionando unos como semilla para formar el poliedro. Tal algoritmo resultaría más eficiente y robusto que otros implementados en la familia Polylla, produciendo mallas de calidad según los criterios de los métodos numéricos.

1.3. Preguntas de investigación

Pregunta 1 ¿Es posible crear poliedros a partir de un algoritmo de generación de poliedros basado en cavidades? ¿Cómo se deberían escoger las semillas para formar los poliedros? ¿Qué pasos de refinamiento se deberían realizar sobre la malla creada?

Pregunta 2 ¿Qué estructura de datos puede proveer un uso eficiente de memoria y tiempo de ejecución permitiendo manejar la topología de la malla? ¿Es posible crear una estructura que permita unir todos los algoritmos dentro de la familia Polylla?

Pregunta 3 ¿El algoritmo final es una solución viable para problemas con mallas típicas? ¿Qué porcentaje de poliedros de la malla final poseen un *kernel*? ¿Se obtiene una reducción sustancial de primitivas? ¿Cuáles son límites que el algoritmo tiene en términos de tamaño de la malla y cantidad de vértices?

Pregunta 4 ¿Es posible paralelizar el algoritmo? ¿Existe una forma directa de hacerlo? ¿Cómo luciría una posible versión paralela?

1.4. Objetivos

El objetivo general de esta investigación es diseñar, implementar y analizar un nuevo algoritmo de creación de malla de poliedro de forma indirecta, inspirado en el uso de cavidades de los tetraedros.

Objetivo 1 Diseñar e implementar un nuevo algoritmo basado en las cavidades de tetraedros semillas.

Objetivo 2 Crear o seleccionar una estructura de datos que sea consistente con un modelo de programación en paralelo.

Objetivo 3 Analizar el desempeño del algoritmo basándose en métricas que permitan evaluar el sobrecosto en memoria y la eficiencia en tiempo de ejecución de un algoritmo paralelo tomando en cuenta su versión secuencial.

Objetivo 4 Estudiar y comparar la calidad de los poliedros contenidos en las mallas generadas con respecto a las versiones actuales de Polylla 3D.

Objetivo 5 Investigar y diseñar una posible versión paralela del algoritmo propuesto.

1.5. Estructura de la Tesis

En el capítulo 2 abordaremos el estado del arte, pasando por otros algoritmos de generación de mallas de poliedros, estructura de datos y terminaremos hablando de una investigación que sirvió de inspiración para el algoritmo desarrollado. En consecuencia, en el capítulo 3 analizaremos en detalle el estado actual de la familia de algoritmos POLYLLA, reconociendo sus fundamentos e analizando la calidad de las mallas generadas. En el capítulo 4 postularemos el algoritmo de cavidades en detalle para finalmente en el capítulo 5 mostrar los resultados de los experimentos y luego concluir en el capítulo 6.

Capítulo 2

Estado del arte

Pauta:

- Tetgen
- Tetwild
- Vorocrust
- Tetgen GPU (cavidades)

En la actualidad las mallas de tetraedros son objeto de estudio de múltiple disciplinas, por su utilidad en gráficos, simulaciones, modelamiento, entre otros. Por otro lado, las mallas de poliedros se han mantenido como algo puramente académico hasta recientemente, con la ayuda de los nuevos métodos numéricos para simulaciones físicas. Estos dos tipos han estado intrínsecamente ligados, por lo tanto, es imperativo conocer, para esta investigación, los algoritmos más relevantes para generar dichas mallas.

2.1. Generación de mallas de tetraedros

Si H [1] planteó un algoritmo secuencial robusto y eficiente que ha servido como base para la generación de tetraedros durante la última década. Su trabajo contiene una sólida justificación teórica e incluye buenas heurísticas prácticas. El algoritmo toma como entrada un *Piecewise Linear Complex* o PLC (Figura 1) para entregar una malla de tetraedros llamada tetraedralización de Delaunay limitada o CDT (*Constrained Delaunay Tetrahedralization*), la cual consiste en una tetraedralización en la que no se exige que se cumpla la condición de Delaunay sobre las primitivas que se encuentran en el borde. El proceso completo consta de 3 etapas: Primero construye la tetraedralización de Delaunay del espacio convexo que cubre el PLC, luego, a partir de esto, crea un CDT en base a los límites o bordes del PLC, y, finalmente, se mejora la calidad de la malla en base a criterios definidos por el usuario.

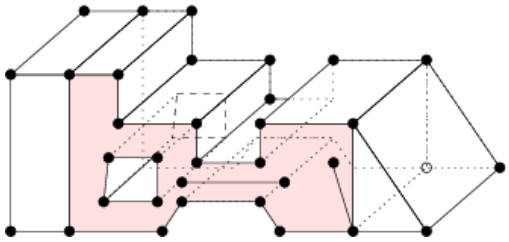


Figura 1: Un PLC es el conjunto de vertices, segmentos y caras que definen la geometría del dominio, extraído de [1] (Figura 1a)

En detalle, la tetraedralización del espacio se realiza mediante dos algoritmos: el de Bowyer-Watson [6, 7] y el *randomized flip* [8], para implementarlos de manera eficiente el autor ordena espacialmente los vértices del *input*. Al momento de construir CDT se puede optar por conservar la superficie del PLC o no, esto se traduce a si el algoritmo tiene permitido o no crear nuevos puntos para alterar la superficie de la malla, en caso de poder hacerlo, la creación de una buena CDT esta garantizada. Para el paso final el algoritmo usa *Delaunay refinement* para conseguir los criterios de calidad dados por el usuario. Los resultados de su investigación culminan en la herramienta TetGen escrita en C++ y disponible gratis en http://ww.tetgen.org.

Dentro de la *Computational Geometry Algorithms Library* [9], se encuentra un algoritmo desarrollado por Alliez P, Jamin C, Rineau L, Tayeb S phane, Tournois J, Yvinec M [10] basado principalmente en el trabajo de Chew [11] y Ruppert [12]. Su solución esta adaptada para trabajar coherentemente con toda la librería, lo cual lo hace una de las más completas, capaz de generar mallas de tetraedros a partir de un *3D Complex* el cual consiste en un conjunto de *caras* con dimensiones 0, 1, 2 y 3 de forma que todas las caras son interiores disjuntos por pares y el límite de cada cara del *complex* es la unión de caras de dimensiones inferiores del complejo. Es un concepto similar al de un PLC donde las dimensiones 0, 1, 2 y 3 son las esquinas, curvas, parches de superficie y sub-dominios, respectivamente.

En líneas generales este algoritmo consiste en un refinamiento de Delaunay iterativo, donde inicialmente se lleva una fase de protección, donde se protegen características de 0 y 1 dimensiones mediante inserciones iniciales y el uso de bolas. Luego del refinamiento, se pasa por una fase de optimización donde se eliminan los *slivers* que corresponden a tetraedros con ángulos muy pequeños y muy grandes, pero que tienen un buen *radius edge ratio* lo que hace que el refinamiento no los elimine.

Por último, Hu Y, Zhou Q, Gao X, Jacobson A, Zorin D, Panozzo D [2] también desarrollaron un algoritmo para construir una malla de tetraedros a partir de un PLC. No obstante, su trabajo se basa en 3 puntos claves: no se presume nada de la forma de la malla de entrada, lo único que se asume es que fundamentalmente siempre contiene errores, por lo que el algoritmo es capaz de alterar su superficie dentro de un ε especificado; la robustez del algoritmo viene primero que todo; y el algoritmo es conservador donde considera

la totalidad de la malla de entrada, incluyendo su superficie y conectividad interna. El algoritmo funciona de la siguiente manera: Primero se parte hace una tetraedralización de Delaunay considerando solo los vértices de la malla, luego se construye un árbol de tipo *Binary Spatial Partition* donde las hojas se corresponde a los triángulos que conforman la superficie del PLC. Con esto el algoritmo trata de mejorar la calidad de la malla teniendo en cuenta la superficie original, haciendo una serie de operaciones locales como *edge collapsing*, *edge splitting*, *face swapping* y *vertex smoothing*. Finalmente, el algoritmo usa la información del BSP para extraer los tetraedros que se encuentran en el interior de la malla y así obtener el resultado final. La robustez se garantiza mediante al uso de una representación exacta de números racionales para los valores, asegurándose de trabajar solo con operaciones que sean cerradas respecto a los racionales. Sumado a esto también existen invariantes que respeta el algoritmo y que el usuario puede ajustar previo a la construcción de la malla. El trabajo de Y. Hu esta disponible en https://github.com/Yixin-Hu/TetWild mediante el nombre de TetWild.

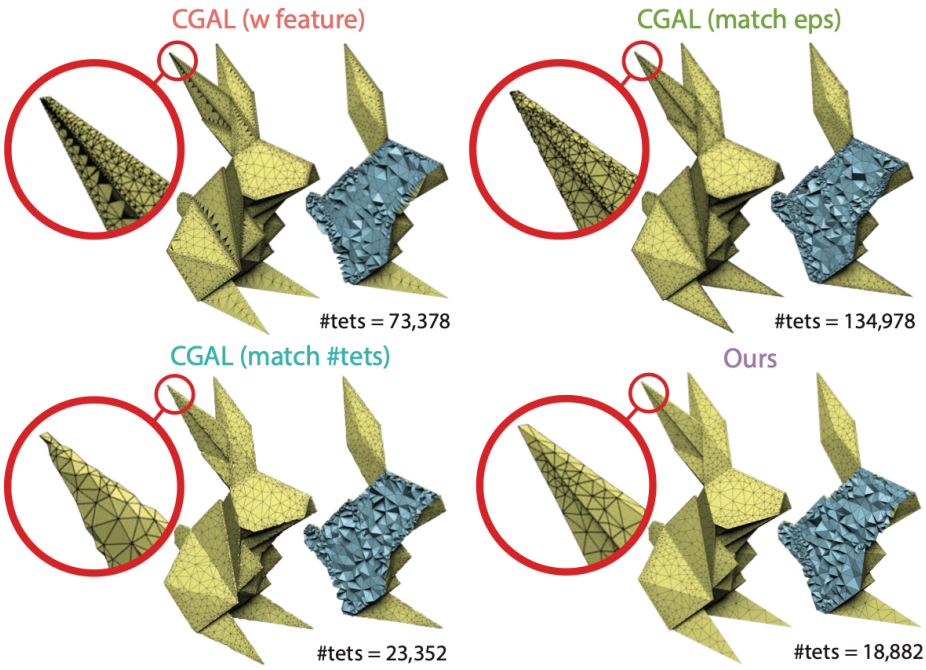


Figura 2: Comparación entre las mallas generados por CGAL con la de TetWild (ours), extraído de [2] (Figura 16)

Chen Z, Tan T-S [13] desarrollaron un algoritmo en GPU basado en [1]. En su trabajo se detallan las consideraciones a la hora de trabajar con una arquitectura *data-parallel*. En particular, el desafío se encuentra al momento de insertar puntos en la malla cuando se debe construir la CDT. Esto debido a que en 3D no existe forma de recuperar el estado antiguo de una malla mediante un *edge flipping*[1] como si se puede hacer en 2D.

[1]Consiste en cuando un par de triángulos *abc* y *cda* comparten una arista *ac* la cual se «voltea» (*flip*) para generar los triángulos *abd* y *bcd* compartiendo ahora *bd*

La solución a esto consiste en usar los conceptos de puntos *Delaunay* independientes y regiones de *Delaunay*. La primera hace referencia a puntos que se encuentran en la misma arista, dichos puntos no pueden ser insertados a la vez de manera concurrente ya que puede provocar que el algoritmo nunca termine. El segundo hace referencia al conjunto de tetraedros cuya bola circunscrita *cubra* un punto p dado y cuyos puntos interiores son visibles para p. Con esto, la inserción de puntos cuyas regiones de *Delaunay* sean disjuntas se puede hacer en paralelo.

Para calcular estas regiones, se construyen cavidades, las cuales consisten en potenciales regiones de *Delaunay*. Se parte inicialmente del punto a insertar y su tetraedro y luego se hace crecer esa cavidad añadiendo los tetraedros vecinos que cumplan con la primera condición de la región de *Delaunay*. Luego se encoge la cavidad para cumplir con la condición de visibilidad dentro de la región. Una vez calculada la región se realiza la inserción en paralelo.

Por otro lado, Kohout J, Kolingerová I, Žára J [14] desarrollaron un algoritmo paralelo basado en un modelo de memoria compartida en CPU. En su trabajo se detallan, además, diversas formas de afrontar el problema de construir una triangulación de Delaunay a partir de una distribución uniforme, donde clasificaron las soluciones en dos tipos: *incremental insert* e *incremental construction*. La primera se basa en partir de una cerradura convexa e ir insertando los puntos interiores a la malla, arreglando los triángulos para cumplir con la condición de Delaunay mediante *edge flipping*. La segunda consiste en partir de una celda única e ir añadiendo puntos adyacentes a la malla.

Para modelos paralelos, la mayor dificultad para la primera solución es la inserción, la cual debe considerar posibles escrituras sobre la misma celda y correcciones que estas conllevan, requiriendo necesariamente de herramientas de sincronización. Para el segundo tipo se encontró que al implementar la solución con un modelo de estilo *divide and conquer*, a la hora de hacer el *merge* este era considerablemente complejo y costoso, por lo que no existía un buen *speed-up*.

Por lo tanto, la solución propuesta por los autores fue un algoritmo llamado *randomized incremental insertion*, como su nombre indica, esta basado en el primer esquema. Es un algoritmo bastante flexible pensado para trabajar con 2-4 procesos en paralelo. Se basa en 3 etapas: "*location* donde una celda en espera a ser dividida se debe encontrar rápidamente, seguido de la *subdivision* y la *legalization* donde se aplica el criterio de la circunferencia y, si es necesario, la localidad de la triangulación es modificada" [14]. Para conseguir esto, el algoritmo trabaja con una grafo acíclico dirigido donde va guardando el historial de cambios de la triangulación.

La solución asume que los posibles *deadlocks* dados por la sincronización entre procesos son poco probable dado la gran diferencia de escala entre la cantidad de procesos (2-4)

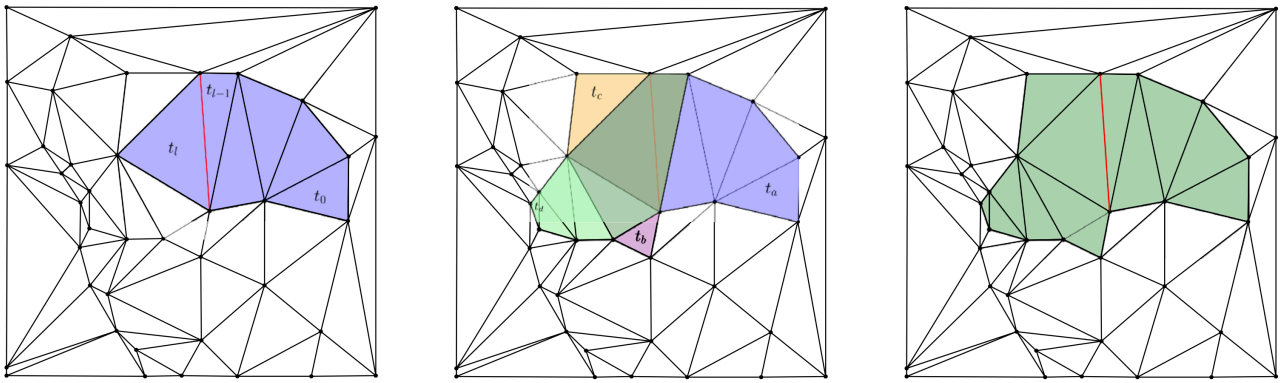
versus la cantidad de vértices (+1000). Cabe destacar que los costos de sincronización y uso operaciones atómicas son bastante más grandes en arquitecturas modernas, las cuales cuentan con una mayor cantidad de procesos.

Sin embargo, el autor destaca en su investigación el algoritmo presentado por [15] como "útil para paralelización masiva" [14]. El algoritmo de Lee F [15] procede de la siguiente manera: primero toma asigna a cada *thread* un punto o más a ser procesado, luego para cada punto se computa su punto más cercano, para asi formar una arista. Luego de esto, secuencialmente un *thread* descarta las aristas repetidas y asigna cada arista única a un *thread*. En paralelo se buscan los dos puntos más cercano a la arista y se construyen 2 triángulos. El proceso continua de esta forma hasta que no queden puntos por procesar.

2.2. Generación de mallas de poliedros

En cuanto a mallas más generales, Abdelkader A, Bajaj CL, Ebeida MS, Mahmoud AH, Mitchell SA, Owens JD, Rushdi AA [16] crearon un algoritmo demostrablemente correcto capaz de generar mallas de poliedros de manera consistente. Basado en los mismos conceptos de la tetraedralización de Delaunay, pero ahora en base a su contra parte que es el diagrama de Voronoi bajo el nombre de VoroCrust. Gracias a esto es capaz de generar mallas de poliedros para dominios generales (convexo, cóncavo y no simples). Funciona generando una malla implícita *dual* dada por un conjunto de semillas de Voronoi y la tetraedralización. En particular, el algoritmo parte generando un conjunto de bolas que describen el borde de la malla o dominio, con las cuales se crean pares de semillas que delimitan el interior y exterior de la malla final. Este malla de la superficie es luego refinada para capturar las *sharp features* de la malla original y entregar una superficie de calidad. Finalmente, se crean bolas para el interior de la malla, partiendo de semillas de Voronoi generadas aleatoriamente.

Recientemente fue publicado el algoritmo conocido como Polylla [3], el cuál es capaz de generar mallas de polígonos de forma arbitraria. Este algoritmo toma como entrada una malla de triángulos y luego va uniendo estas primitivas para formar polígonos arbitrarios, pudiendo ser convexo como no convexos. La unión se hace a través de las aristas más largas de cada triángulo. Actualmente existe una versión paralelizada en GPU de este algoritmo [17]. Para construir los polígonos, Polylla usa el concepto de *Longest-edge propagation path* (Lepp) [18] (lista ordenadas de triángulos $t_0, t_1, ..., t_n$, tales que t_i es vecino de t_{i-1} por la arista más larga de t_{i-1}, para todo $i = 1, 2, ..., n$), y la de *Terminal-edge region* [3] como la region R formada por la union de todos los triángulos t_i cuyos Lepp tengan el mismo *terminal-edge* (consiste en la arista compartida entre 2 triángulos que a la vez es la más larga de ambos). Polylla va generando polígonos (Figura 3) a partir de estas regiones de manera antihoraria. Esto suele producir polígonos degenerados que deben ser revisados una vez generada la malla.



(a) (b) (c)

Figura 3: Creación de una *Terminal-edge region*

2D Polylla convierte *Terminal-edge regions* en polígonos, este proceso hace uso de las *terminal-edge* que pueden definirse tanto en 2D como en 3D [18], con esto se puede desarrollar un generador de mallas de poliedros. Adicionalmente, con esta nueva dimensión, podemos extender el concepto de *terminal-edge* a las caras de los tetraedros, creando así las *Terminal-face region*, y luego estas regiones transformarlas en poliedros [5]. Por lo tanto, dada una malla de tetraedros $\tau = (V, E, F)$ y un criterio de unión J, podemos diseñar este algoritmo de dos maneras:

2.3. Estructura de datos

2.4. Cavidades

Capítulo 3

Trabajo anterior

El trabajo propuesto forma parte de una familia mayor de algoritmos llamados Polylla, los cuales se basan en la idea de unir primitivas bajo un criterio geometrico. En particular, actualmente existe una versión bidimensional el cual construye polígonos, y extensiones a tres dimesiones para crear poliedros.

Pauta:

- Polylla 2D
- Polylla 3D Face y Edge
- Criterios investigados
- Experimentos y resultados

3.1. POLYLLA 2D

3.2. POLYLLA 3D

3.3. Criterios de unión

Capítulo 4

Algoritmo de cavidades

Deberia mencionar la intención de unir todo el ecosistema como una sola libreria? Que tal la herramienta para testear la calidad de las mallas? Como se observa, en tres dimensiones, podemos caracterizar los algoritmos por la forma en la que unen las primitivas y los criterios utilizados para distinguir que unión es mejor con respecto a otra. Polylla Edge y Face corresponde a una clasificación en la forma, mientras que los criterios de unión corresponden a una clasificación de segundo tipo. Con esto en mente podemos generalizar el algoritmo Polylla 3D

4.1. Arquitectura

4.2. Visualización

Capítulo 5

Experimentos y resultados

Capítulo 6

Conclusión

Bibliografía

[1] H. Si, «TetGen, a Delaunay-Based Quality Tetrahedral Mesh Generator», *ACM Transactions on Mathematical Software*, 2015, doi: 10.1145/2629697.

[2] Y. Hu, Q. Zhou, X. Gao, A. Jacobson, D. Zorin, y D. Panozzo, «Tetrahedral meshing in the wild», *ACM Transactions on Graphics*, 2018, doi: 10.1145/3197517.3201353.

[3] S. Salinas-Fernández, N. Hitschfeld, A. Ortiz-Bernardin, y H. Si, «POLYLLA: polygonal meshing algorithm based on terminal-edge regions», *Engineering With Computers*, 2022, doi: 10.1007/s00366-022-01643-4.

[4] L. B. da Veiga, F. Brezzi, A. Cangiani, G. Manzini, L. D. Marini, y A. Russo, «BASIC PRINCIPLES OF VIRTUAL ELEMENT METHODS», *Mathematical Models and Methods in Applied Sciences*, 2012, doi: 10.1142/s0218202512500492.

[5] M. Alvarez, «Generación de mallas de poliedros: optimización y evaluación», Tesis de Magíster, 2024.

[6] D. F. Watson, «Computing the n-dimensional Delaunay tessellation with application to Voronoi polytopes», *The Computer Journal*, 1981, doi: 10.1093/comjnl/24.2.167.

[7] A. Bowyer, «Computing Dirichlet tessellations», *The Computer Journal*, 1981, doi: 10.1093/comjnl/24.2.162.

[8] H. Edelsbrunner y N. R. Shah, «Triangulating topological spaces», en *Proceedings of the Tenth Annual Symposium on Computational Geometry*, 1994. doi: 10.1145/177424.178010.

[9] The CGAL Project, *CGAL User and Reference Manual*, 5.6. CGAL Editorial Board, 2023. [En línea]. Disponible en: https://doc.cgal.org/5.6/Manual/packages.html

[10] P. Alliez, C. Jamin, L. Rineau, S. phane Tayeb, J. Tournois, y M. Yvinec, «3D Mesh Generation», en *CGAL User and Reference Manual*, 5.6., CGAL Editorial Board, 2023. [En línea]. Disponible en: https://doc.cgal.org/5.6/Manual/packages.html#PkgMesh3

[11] L. P. Chew, «Guaranteed-quality mesh generation for curved surfaces», 1993. doi: 10.1145/160985.161150.

[12] J. Ruppert, «A Delaunay Refinement Algorithm for Quality 2-Dimensional Mesh Generation», *Journal of Algorithms*, 1995, doi: 10.1006/jagm.1995.1021.

[13] Z. Chen y T.-S. Tan, «Computing Three-Dimensional Constrained Delaunay Refinement Using the GPU», en *2019 28th International Conference on Parallel Architectures and Compilation Techniques (PACT)*, 2019. doi: 10.1109/pact.2019.00039.

[14] J. Kohout, I. Kolingerová, y J. Žára, «Parallel Delaunay triangulation in E2 and E3 for computers with shared memory», *Parallel Computing*, 2005, doi: 10.1016/j.parco.2005.02.010.

[15] F. Lee, «Constructing the constrained Delaunay triangulation on the Intel paragon», en *Proceedings of the 1997 ACM Symposium on Applied Computing*, 1997. doi: 10.1145/331697.332333.

[16] A. Abdelkader *et al.*, «VoroCrust», *ACM Transactions on Graphics*, 2020, doi: 10.1145/3337680.

[17] S. Salinas, J. Ojeda, N. Hitschfeld, y A. Ortiz-Bernardin, «GPU Parallel algorithm for the generation of polygonal meshes based on terminal-edge regions». [En línea]. Disponible en: http://arxiv.org/abs/2204.05438

[18] M.-C. Rivara, «New longest-edge algorithms for the refinement and/or improvement of unstructured triangulations», *International journal for numerical methods in Engineering*, 1997.